

Benchmark Radar: AI Benchmark Discovery with Traceable Evidence

Koutian Wu^{1,2} * Junjie Zhou⁴ Ergan Shang³ Jiayu Wang⁵ Pengqian Han⁶

¹ Earth and Space AI ² Tacite AI ³ Carnegie Mellon University

⁴ Hangzhou Dianzi University ⁵ Xi'an Jiaotong University

⁶ The University of Auckland

Abstract

Selecting an AI benchmark requires evidence about its tasks, artifacts, and evaluation settings as well as model scores. We present Benchmark Radar, a searchable catalog that keeps a separate entry for each source’s benchmark record and attaches its scores, model identities, and cited documents. This design preserves records with missing measurements and lets readers inspect disagreements between sources. At the 2026-09-07 cutoff, a census of 1,283 records across 4 sources finds 12,916 numeric score observations on 790 records. Of the 493 unscored records, 464 still link to a paper, repository, or dataset. Requiring a score for discovery would discard these routes to prior work. The census also separates model coverage from document coverage and identifies missing scale and date information that limits score comparisons. A contributor’s prior-art search illustrates how to use the evidence when assessing candidates. We release the catalog, web and offline query interfaces, and a reproducible census with record-level provenance.

1,283 records. 790 with scores. 493 without.

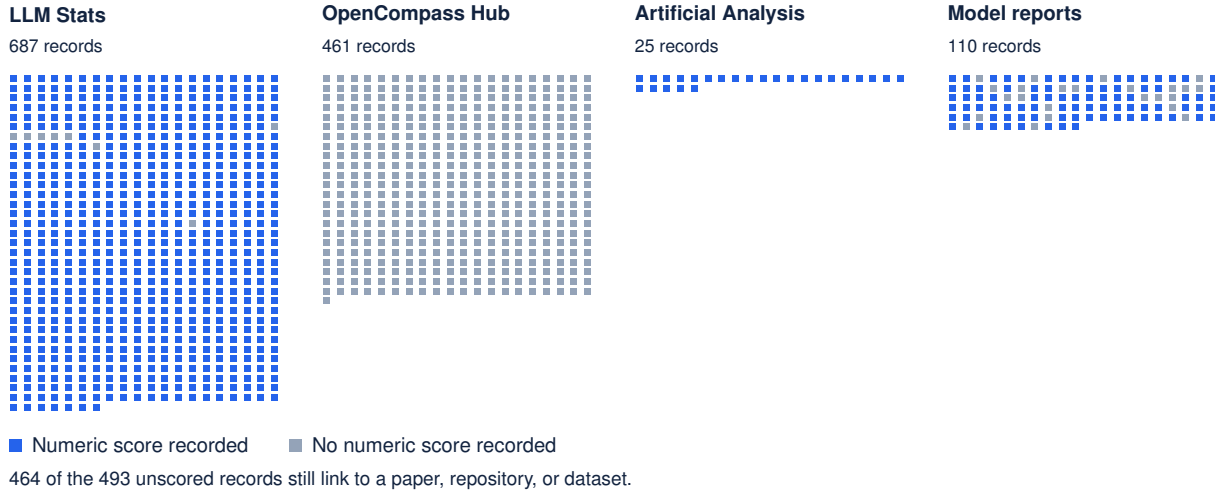


Figure 1: Discovery extends beyond scored records: 464 unscored entries still provide artifact links. Each mark represents one source record and links to its detail page. All 1,283 records appear, without a date or score filter.

*Corresponding author: k@tacite.ai

1 Introduction

A researcher choosing or designing an AI evaluation needs to inspect what a benchmark tests, where its data and code reside, and how reported scores were obtained. Those decisions require more than a benchmark name or leaderboard position. A record can offer useful task materials without an archived score, while similarly named records can describe different versions or evaluation settings.

Combining benchmark sources creates two linked problems. Requiring complete measurements can hide records that remain useful for discovery. Combining scores or counts across related names can erase distinctions needed to interpret the evidence. We study how to make benchmark records searchable together while retaining the information needed to judge each candidate.

Benchmark Radar preserves one record per source benchmark and attaches its observations and citations. Readers can follow reviewed links between related records, but each record retains its own scores and model and document counts. General search includes records with missing measurements. A comparison uses the subset with the measurements it requires and states that coverage.

We contribute a catalog and query system that exposes this evidence through shared web and offline interfaces, and a reproducible census of the full catalog. The census tests what a score requirement would remove from discovery, distinguishes model coverage from document coverage, and measures the availability of score scales and dates. Its main finding is that 464 unscored records still link to task materials (Figure 1). These links justify keeping those records available for inspection; they do not establish that a benchmark suits a particular research question. Appendix C illustrates that final judgment in a contributor’s prior-art search.¹

2 Related Work and Scope

Benchmark catalogs and evaluations. LLM Stats publishes benchmark descriptions and reported model results [9]. OpenCompass provides an evaluation platform and a benchmark registry with artifact metadata [11]. Artificial Analysis publishes model evaluations and their methodology [1]. Benchmark Radar uses records from these sources alongside model-report evidence. Its contribution is the shared, inspectable record structure and access across sources; we do not claim improvements to their evaluation methods or more accurate model rankings.

Documenting evaluation evidence. Model cards and datasheets motivate documenting evaluation conditions and dataset characteristics [4, 10]. Benchmark Radar retains that information when the collected sources provide it, with citations for follow-up review. Benchmarks such as GPQA [13], SWE-bench [7], and SciBench [16] define different tasks; placing their records in one catalog does not make their scores interchangeable. The census measures which evidence is available for inspection and which metadata still needs review.

3 Method

3.1 Preserve Records Before Comparing Measurements

A source record is one benchmark entry from one contributing source. We normalize names, identifiers, artifact links, score observations, model identities, and cited documents into common fields. A record remains in the catalog if a score, date, or citation is absent. Reviewed identity links connect related

¹Source code: <https://github.com/ktwu01/benchmark-radar>.

records while preserving their separate observations and counts. A similar name alone does not authorize a merge.

Daily discovery contributes a different kind of evidence: collected mentions, releases, and updates. Exact identifiers such as DOIs, arXiv IDs, and repository URLs link those observations to artifacts. Discovery observations do not increase the benchmark catalog total. Appendix A describes the collection and publication paths; Appendix B records source health at the cutoff.

3.2 Retrieve Candidates with Their Evidence

The same benchmark IDs reach web search, detail pages, dataset exports, and offline clients. Lexical search uses BM25F, a field-weighted word-matching score [14], with bounded boosts for name and phrase matches. Each result exposes matched and missing query words, the fields they occur in, and the score components. Source membership does not change the ranking. A shared query service supplies the CLI and HTTP interfaces with the same response format and local data provenance.

Candidate retrieval precedes suitability judgment. An analyst or agent can try focused query variants, inspect a record’s tasks and score settings, and follow its citations. The interface preserves the evidence needed for that review. This paper evaluates catalog and measurement coverage; the contributor case illustrates usage without measuring retrieval accuracy or time saved.

3.3 Audit the Full Population

The census starts with every record in the rebuilt catalog index and reads its detail file. It applies no date, score, or interface filter. We count finite numeric score observations once by observation ID. Within each benchmark record, we count scored models by source model ID, preserving separately evaluated configurations, and cited documents by document ID. Repeated observations do not create additional models or documents. The global model registry uses its recorded identity links; per-benchmark model counts are not summed into a global total.

Eligibility depends on the measurement a calculation needs. A declared percentage unit, known score direction, and numeric values within 0–100 permit a percentage-scale summary. Rescaling a displayed value or reading an aggregator’s declared maximum does not establish that unit. Matching scales also do not establish matching test versions, prompts, tools, attempts, or evaluators.

Date coverage counts valid recorded benchmark release dates. Score entries retain their own date basis, including model announcements and document publication. We do not substitute those dates for an evaluation date. Appendix D provides the software revision, input hashes, census script, and validation procedure.

4 Results

4.1 Unscored Records Still Provide Routes to Prior Work

The rebuilt catalog contains 1,283 source records across 4 sources. We found 12,916 numeric score observations on 790 records, with 493 records lacking numeric scores (Table 1). These are source-record counts: two sources can describe a related benchmark, and we retain both records without claiming they represent distinct underlying tests.

Source	Records	Scored	Unscored	Numeric scores
LLM Stats	687	679	8	5,544
OpenCompass Hub	461	0	461	0
Artificial Analysis	25	25	0	7,050
Model reports	110	86	24	322
Total	1,283	790	493	12,916

Table 1: Score coverage across the full catalog: 790 records have numeric observations and 493 do not. The zero in the OpenCompass score column describes this snapshot, not whether those benchmarks have ever been evaluated. Scores count observations, not distinct models.

Of the 493 unscored records, 464 have at least one paper, repository, or dataset link. Across the full catalog, 475 records have a paper link, 506 have a repository link, and 293 have a dataset link; the categories overlap. For example, the OpenCompass Hub record for A-Bench links a paper, code repository, and dataset despite having no archived score. These links give researchers routes to inspect tasks and baselines before deciding whether an evaluation fits their work. Link presence does not establish current availability or permission to use an artifact.

4.2 Document Counts Do Not Measure Model Coverage

The common document registry contains 1,208 distinct cited documents, attached to 1,278 of the 1,283 benchmark records. Five records have no cited documents. The shared model registry contains 868 model identities. Within the catalog, all 790 scored records have sufficient model IDs to count distinct models; unscored records have unknown scored-model coverage.

Table 2 illustrates why these counts cannot substitute for one another. The Artificial Analysis record for GPQA Diamond contains scores for 586 models and cites one registry page. The Model reports record cites 27 documents and contains 21 numeric observations for 19 models. Both describe useful evidence, but adding their counts would require reviewed model and evaluation identities. Neither row alone measures adoption across the field.

Benchmark record	Source	Models	Documents	Scores
GPQA Diamond	Artificial Analysis	586	1	586
GPQA Diamond	Model reports	19	27	21
Humanity’s Last Exam	Artificial Analysis	577	1	577
SciCode	Artificial Analysis	577	1	577
CritPt	Artificial Analysis	492	1	492

Table 2: One cited page can document hundreds of scored models. Models count distinct source model IDs within a record; documents count distinct citation IDs; scores count numeric observations. These selected examples illustrate the units. The census retains all source records.

Table 2 also records 577 scored models for Humanity’s Last Exam and SciCode, and 492 for CritPt, each under one Artificial Analysis citation. Reading that citation count as model coverage would hide the measured breadth of those records.

4.3 Numeric Scores Often Lack a Verified Common Scale

Only 82 of the 790 scored records declare a percentage unit with a known direction and values within 0–100. The remaining 708 scored records use other or unverified scales. We retain their numeric observations, which do not support a shared percentage-headroom calculation. Table 3 accounts for the entire catalog before any such comparison.

Measurement state	Records	Interpretation
Numeric scores, declared percentage scale	82	Supports scale-specific summaries; protocols still need checking.
Numeric scores, other or unverified scale	708	Retain scores; do not assume a 100-point ceiling.
No numeric score	493	Score headroom remains unknown.
Full population	1,283	Every source record remains in the census.

Table 3: Only 82 of 790 scored records meet the percentage-scale rule in Section 3.3. All 1,283 records remain accounted for. A declared scale alone does not establish a common evaluation protocol.

The catalog records benchmark release dates for 615 records, leaving 668 without a release date. Across score observations, 12,594 entries carry model-announcement dates and 322 carry document-publication dates. Neither date basis directly establishes an evaluation date. This archive therefore cannot support a catalog-wide claim about score stagnation from a gap between model-report mentions and score-entry dates. Researchers must first establish dates and comparable evaluation settings for the relevant source records.

A small gap to a metric ceiling can describe a reported setup. It does not show that a benchmark has exhausted its ability to distinguish models across versions and settings. Catalog-wide headroom or recency estimates require the eligible measurement coverage alongside the statistic.²

5 Limitations and Future Work

The census describes this catalog at its recorded cutoff. Collection limits, failed requests, missing identifiers, and different snapshot dates affect its coverage. Source records are not a count of distinct underlying tests. Artifact links provide routes for inspection but do not verify present availability or licensing. Broader coverage requires additional source collection and review of the evidence already present.

The study does not measure retrieval precision, task suitability, or time saved. The worked example combines catalog queries with web search and has no controlled baseline. Lexical matching can miss paraphrases and renamed tasks. Evaluating semantic retrieval [12] will require reviewed relevance judgments across queries and candidate records, including less familiar benchmark families.

The scale and date audit identifies metadata gaps rather than benchmark saturation or score stagnation. Establishing those outcomes requires source review for comparable test versions and settings, and dates tied to actual score reporting or evaluation. Matching archive labels cannot verify independent repeated runs.

Task-capability classification also remains unvalidated in this rebuild. The deterministic null extractor used in CI assigns no capability levels to its 5,863 discovery-derived tracks. Completing and evaluating those labels is a separate task from maintaining the benchmark catalog.

²Junjie Zhou contributed a score-archive audit and earlier report revisions, documented in [issue #457](#).

6 Conclusion

Benchmark Radar makes benchmark discovery and evidence inspection available through one catalog while retaining each source’s records and citations. The full-catalog census shows a concrete cost of requiring scores for discovery: 464 unscored records with artifact links would disappear. It also shows why model counts, document counts, and score observations need separate definitions, and why numeric values alone do not establish comparability. Researchers can use the catalog to identify candidates and inspect the evidence before choosing an evaluation. Further work should test retrieval quality and improve the missing measurement metadata across sources.

Author note. Koutian Wu led the work and manuscript preparation, built the initial collection pipeline, and implemented data aggregation across sources. Ergun Shang prepared figures and contributed copyediting. Pengqian Han contributed copyediting.

References

- [1] Artificial Analysis. Artificial analysis intelligence benchmarking methodology, 2026. URL <https://artificialanalysis.ai/methodology/intelligence-benchmarking>. Accessed 2026-09-06.
- [2] arXiv. arXiv API User Manual, 2026. URL <https://info.arxiv.org/help/api/user-manual.html>. Accessed 2026-09-06.
- [3] Citation File Format developers. Citation file format 1.2.0, 2021. URL <https://citation-file-format.github.io/>.
- [4] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.
- [5] GitHub Docs. REST API Endpoints for Search, 2026. URL <https://docs.github.com/en/rest/search/search>. Accessed 2026-09-06.
- [6] Hugging Face. Hugging Face Hub Documentation, 2026. URL <https://huggingface.co/docs/hub/index>. Accessed 2026-09-06.
- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2023. URL <https://arxiv.org/abs/2310.06770>.
- [8] Daniel S. Katz et al. Recognizing the value of software: a software citation guide. *F1000Research*, 9:1257, 2021. doi: 10.12688/f1000research.26932.2. URL <https://doi.org/10.12688/f1000research.26932.2>.
- [9] LLM Stats. AI and LLM benchmarks 2026: Rankings, scores and results, 2026. URL <https://llm-stats.com/benchmarks/>. Accessed 2026-09-06.
- [10] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.
- [11] OpenCompass Contributors. Dataset statistics, 2026. URL https://opencompass.readthedocs.io/en/latest/dataset_statistics.html. Accessed 2026-09-06.
- [12] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019. doi: 10.18653/v1/D19-1410.

- [13] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2023. URL <https://arxiv.org/abs/2311.12022>.
- [14] Stephen Robertson, Hugo Zaragoza, and Michael Taylor. Simple BM25 extension to multiple weighted fields. In *Proceedings of the Thirteenth ACM International Conference on Information and Knowledge Management*, 2004. doi: 10.1145/1031171.1031181.
- [15] Arfon M. Smith, Daniel S. Katz, and Kyle E. Niemeyer. Software citation principles. *PeerJ Computer Science*, 2:e86, 2016. doi: 10.7717/peerj-cs.86. URL <https://doi.org/10.7717/peerj-cs.86>.
- [16] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. SciBench: Evaluating college-level scientific problem-solving abilities of large language models. *arXiv preprint arXiv:2307.10635*, 2023. URL <https://arxiv.org/abs/2307.10635>.
- [17] L. Xiaopai. Builderpulse: Ai-powered daily intelligence for indie hackers and builders. GitHub, 2026. URL <https://github.com/BuilderPulse/BuilderPulse>.

Appendix

A Collection and Reader Interfaces

Benchmark Radar adapts BuilderPulse’s daily public-source collection approach [17]. Figure 2 separates the benchmark catalog from discovery history. Model reports include model cards, technical reports, system cards, and release posts; they contribute through the same record structure as benchmark registries.

A.1 Discovery Collection

Daily discovery observations describe mentions, releases, and updates found across public sources. An observation is one collected record; an artifact is a paper, repository, dataset, release, or page linked by exact identifiers. These objects differ from source-specific benchmark records. We retain their histories alongside the catalog without adding discovery observations to the benchmark total.

Each collection run searches a 48-hour window, records counts and errors by source, removes future-dated rows, and requires healthy core sources before publication. At the cutoff, arXiv [2], Hugging Face Hub [6], and GitHub Search [5] were the core sources. Additional routes cover scholarly indexes, dataset hosts, repository releases, and institutional feeds. Appendix B records their cutoff status.

The 2026-09-07 snapshot records 1,003 fetched rows and 954 candidates after duplicate removal. Of these, 366 qualified for publication and 138 met the recommendation threshold. These are discovery-run counts, separate from the benchmark catalog census. Published evidence retains source URLs, retrieval times, parser versions, identifiers, and payload checksums. Raw responses and credentials remain outside the public artifact.

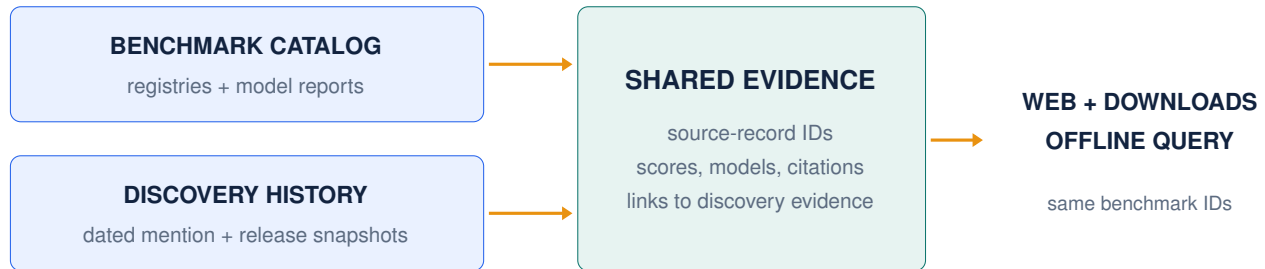


Figure 2: Two input paths support publication: dated discovery snapshots preserve mention and release evidence, while benchmark registries and model reports populate the shared catalog. Clients use the catalog’s stable source-record IDs.

A.2 Reader Interfaces and Display Filters

Table 4 describes the interfaces. Search and exports retain the full catalog; display filters affect only the selected view.

Benchmark Frontier, on the Leaderboard tab, requires a numeric score and excludes known pre-2024 benchmarks. It retains scored records with unknown dates or counts through labelled marks, and keeps unverified scales outside Pareto calculations. Its initial score cutoff is 70. The linked score

Interface	Reader question	Evidence and scope
Today	What appeared or changed?	Daily discovery observations, source health, and links.
Search	Which benchmarks match this task?	The complete catalog, including unscored records.
Leaderboard	Which evaluations combine lower scores with broader measured use?	Scored source records; distinct scored models or cited documents supply the selected count.
Saturation	What scores and settings were reported?	Benchmark browsing and complete score histories; search reaches every source and year.
CLI and HTTP	Can I inspect the same evidence locally?	One query service and the same benchmark IDs and response format.

Table 4: Reader questions and evidence scope. A search result is a candidate for inspection, not a suitability judgment.

ranking retains its date and numeric-score requirements independently of that cutoff. General catalog search and exports retain the full population. The census in this paper applies none of these display filters.

A separate priority score for daily recommendations combines relevance (35%), evidence (20%), recency (20%), and adoption (25%) on a 0–100 scale. These weights describe reading priority for discovery observations. An agent assessing benchmark suitability can try focused query variants, inspect record details, and read the cited sources; that judgment occurs after candidate retrieval.

A.3 Discovery Coverage Alongside the Catalog

The daily discovery collection contains 11,068 observations and 6,546 artifacts linked by exact identifiers across 46 snapshots. Four snapshots are simulated historical backfills; their dates describe reconstructed collection windows. These discovery units remain separate from benchmark records.

Five discovery source labels account for 9,743 of 11,068 observations (88.0%). Figure 3 includes the remaining labels in one aggregate bar. Source caps and collection failures can affect this mix, so it does not estimate the distribution of benchmark research. Fifty-nine of 6,546 artifacts have observations from multiple sources; missing or inconsistent identifiers can leave further relationships unresolved.

B Source Inventory and Health

Table 5 lists source connectors that completed successfully at the cutoff. Row counts are measured before duplicate removal and ranking. A connector can be healthy with zero eligible rows if its request succeeds and the response can be parsed. Core sources cover papers, shared model or dataset artifacts, and code; all must be healthy before publication. Failures in optional sources are reported but do not block publication.

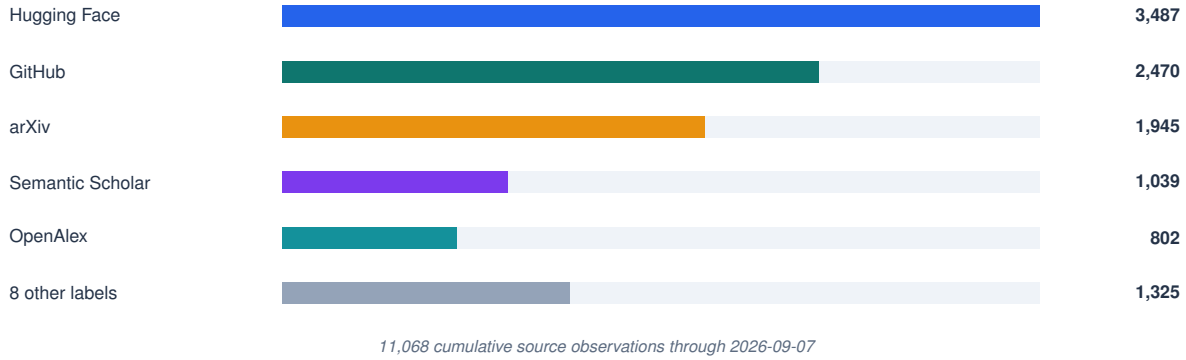


Figure 3: Discovery observations by source through 2026-09-07. Repeat sightings count as observations; these bars do not count benchmark records.

Connector	Role	Cutoff status	Core
arXiv	Primary papers via selected AI, language, vision, and software categories	Healthy; 46 rows	Yes
Hugging Face Hub	Datasets and Spaces connected to benchmark artifacts	Healthy; 101 rows	Yes
GitHub Search	Code repositories and benchmark artifacts	Healthy; 300 rows; at cap	Yes
GitHub Organizations	Reviewed organization repositories	Healthy; 6 rows	No
Hugging Face Papers	Community-surfaced papers	Healthy; 26 rows	No
Kaggle Datasets	Public benchmark datasets	Healthy; 30 rows	No
Zenodo	DOI-bearing research artifacts	Healthy; 65 rows	No
Crossref	DOI metadata	Healthy; 180 rows	No
OpenReview	Conference submissions	Healthy; no eligible rows	No
GitHub Releases	First-party releases	Healthy; 1 row	No
OpenAlex	Scholarly discovery metadata	Healthy; 248 rows	No

Table 5: Healthy direct discovery connectors at the current cutoff.

Table 6 lists research and engineering feeds operated by the institutions being monitored. They can surface benchmark papers, datasets, model announcements, and engineering posts. The feed collector returned no records at the cutoff. Semantic Scholar returned a malformed payload, and Brave Search lacked an API key. These optional-route failures remain recorded in the snapshot. Searches restricted to an organization’s website cover institutions that lack a verified feed.

Feeds 1-12	Feeds 13-24
Meituan Engineering	Ai2
OpenAI News	Together AI
Google AI	Sakana AI
Google DeepMind	Qwen
Google Research	Ollama
Apple Machine Learning Research	Stability AI
AWS Machine Learning	Nomic AI
Hugging Face Blog	Replicate
Microsoft Research	NVIDIA Developer
NVIDIA AI Blog	IBM Research
Mistral AI	Databricks
Meta Research	LangChain

Table 6: First-party research and engineering feeds monitored by Benchmark Radar.

C Worked Use Case: Prior-Art Check for a New Evaluation

The contributor wanted to determine whether a proposed evaluation duplicated existing work. During August 2026, they surveyed recent work on credit assignment in agentic training, using small Qwen-series models as a requirement for reproducible baseline experiments. The search produced a focused comparison table that the contributor used to assess the proposed design.

The contributor gave the task to a coding agent together with the public setup instructions for Benchmark Radar. The agent installed the command-line client and the Benchmark Radar Skill, a set of instructions for using the client. It then downloaded the local corpus and queried candidate records offline.

```

what can i do with benchmark-radar cli?

+ Thought: 328ms

$ benchmark-radar --help
usage: benchmark-radar [-h] [--author-output AUTHOR_OUTPUT]
                        [--author-contacts AUTHOR_CONTACTS]
                        [--author-repo-limit AUTHOR_REPO_LIMIT]
                        [--author-contributors AUTHOR_CONTRIBUTORS]
                        [--author-emails] [--config CONFIG] [--output OUTPUT]
                        [--json-output JSON_OUTPUT]
                        [--social-output SOCIAL_OUTPUT] [--items ITEMS]
                        [--repo REPO] [--channels CHANNELS]
                        [--existing-body EXISTING_BODY] [--since SINCE]
                        [--until UNTIL] [--snapshot-dir SNAPSHOT_DIR]
--
Click to expand

+ Thought: 531ms

The CLI has two groups:

Consumer (query) commands - for searching and inspecting local benchmark data:
- search "query" --scope catalog|radar|all --json - discover benchmarks (catalog = verified records, radar = recent unverified evidence like papers/repos/datasets)
- show "key-or-slug" --json - inspect one record in detail
- recent --json - newest Radar evidence
- status --json - local data health/provenance
- init / sync - set up or refresh the local data
- serve --host 127.0.0.1 --port 8765 - local HTTP interface for the same queries

Maintainer commands - for operating the radar pipeline (not for ordinary use): run, rebuild, backfill, migrate, rescore, simulate-history, export, classify, authors, social, normalize-external, build-data-release.

For your local benchmark search use case, you'll mostly need search, show, recent, status, and serve.

Build · DeepSeek V4 Flash DeepSeek
..... esc Interrupt 150.1K (15%) · $0.03 ctrl+p commands

```

Figure 4: A coding agent installs the Benchmark Radar client and its usage instructions, then runs local queries.

The agent inspected paper, repository, and dataset links to decide which sources to open next. Figures 5 and 6 show two retrieved records: one has a repository link, and the other has both paper and repository links. Neither records a dataset link, leaving dataset availability for a follow-up check.

```

    "rank": 18,
    "recommended": false,
    "score": 18.75,
    "slug": null,
    "snapshot_date": "2026-09-01",
    "source": "Hugging Face Papers",
    "source_id": "2608.28476",
    "updated_at": "2026-08-28T00:00:00+00:00",
    "url": "https://huggingface.co/papers/2608.28476"
  },
  {
    "categories": [
      "benchmark"
    ],
    "description": "LTABC-KM is a long-tail-aware adaptive artificial bee colony-K-means for unsupervised image clustering. It leverages density-aware coreset and graph-propagation assignment to recover minority tail modes without labels, achieving significant Macro-F1, NMI and ARI improvements on imbalanced benchmarks with favorable quality-computation trade-offs.",
    "has_dataset": false,
    "has_paper": false,
    "has_repo": true,
    "has_size": false,
    "key": "radar:github:Lutra11/LTABC-KM",
    "kind": "radar",
    "languages": [],
    "match": {
      "idf_coverage": 0.4391,
      "matched_fields": [
        "description"
      ],
      "matched_tokens": [
        "assignment"
      ],
      "missing_tokens": [
        "credit",
        "training"
      ],
      "phrase_fields": [],
      "query_coverage": 0.3333,
      "ranking_algorithm": "bm25f_v3",
      "reason": "1 of 3 unique query tokens matched across fields; IDF coverage 0.44",

```

"has_dataset": false,
 "has_paper": false,
 "has_repo": true,
 "has_size": false,

```

    "key": "radar:github:Lutra11/LTABC-KM",
    "kind": "radar",
    "languages": [],
    "match": {
      "idf_coverage": 0.4391,
      "matched_fields": [
        "description"
      ],
      "matched_tokens": [
        "assignment"
      ],
      "missing_tokens": [
        "credit",
        "training"
      ],
      "phrase_fields": [],
      "query_coverage": 0.3333,
      "ranking_algorithm": "bm25f_v3",
      "reason": "1 of 3 unique query tokens matched across fields; IDF coverage 0.44",

```

□
Build · DeepSeek V4 Flash DeepSeek

D:\projects\radar-test
187.3K (19%) · \$0.04 **ctrl+p** commands

Figure 5: A candidate record with a repository link for inspecting the implementation. Paper and dataset links are missing from the record.

```
"source": "GitHub",
"source_id": "mdkamranalam/credixa",
"updated_at": null,
"url": "https://github.com/mdkamranalam/credixa"
},
{
  "categories": [
    "benchmark"
  ],
  "description": "Long-horizon agentic tasks require large language models (LLMs) to iteratively retrieve, integrate, and maintain dispersed information across multi-turn interactions, but preserving all interaction histories leads to a continuously growing working context. Recent proactive context management methods allow models to edit their own working context with specialized tools, yet they still face three key limitations: (1) a limited toolset restricted to search, deletion, and summarization, with no support for global planning, long-term memory, and adaptive compression; (2) inefficient exploration that treats context management actions uniformly despite their heterogeneous impacts on final outcomes; and (3) coarse-grained credit assignment that assigns the final trajectory-level reward to all intermediate context editing actions during RL. To bridge these gaps, we introduce ContextPilot, a proactive context management framework for long-horizon agentic reasoning. Our approach systematically augments the toolset with planning, long-term memory, and soft context offloading tools. We further propose an RL method tailored for context management, which uses context and entropy variation to identify critical editing decisions for branch sampling and estimates action-level advantages from all branched trajectories that pass through the corresponding context editing action. Experiments on long-context QA and deep search tasks show that ContextPilot achieves stronger performance with a more compact working context, consistently outperforming existing baselines across various base models and benchmarks. Code is available at https://github.com/Tencent/ContextPilot.",
  "has_dataset": false,
  "has_paper": true,
  "has_repo": true,
  "has_size": false,
  "key": "radar:hugging face papers:2608.28476",
  "kind": "radar",
  "languages": [],
  "match": {
    "idf_coverage": 0.8392,
    "matched_fields": [
      "description"
    ],
    "matched_tokens": [
      "assignment",
      "credit"
    ]
  },
}
```

Build · DeepSeek V4 Flash DeepSeek

D:\projects\radar-test187.3K (19%) · \$0.04ctrl+p commands

Figure 6: A second candidate provides both paper and repository links. Its dataset link needs a follow-up check.

The agent supplemented catalog queries with web search to find related work described in different terms. It cross-checked the candidate sets and inspected the source evidence before selecting records for the comparison. Figure 7 shows the word matches and score components available for inspecting one retrieved candidate.

```

"kind": "radar",
"languages": [],
...
Click to expand

$ benchmark-radar search "credit assignment training" --scope radar --json

...
"query_coverage": 0.3333,
"ranking_algorithm": "bm25f_v3",
"reason": "1 of 3 unique query tokens matched across fields; IDF coverage 0.44",
"retrieval_score": 10.824384,
"score_components": {
  "bm25f": 10.824384,
  "name_bonus": 0.0,
  "phrase_bonus": 0.0
},
"modality": null,
"name": "deepvisionaiofficial/Automation_AI-Assignment-Checker",
"openness": null,
"published_at": "2026-08-10T05:56:17+00:00",
"publisher": "",
"rank": 6,
"recommended": false,
"score": 36.32,
"slug": null,
"snapshot_date": "2026-08-10",
"source": "GitHub",
"source_id": "deepvisionaiofficial/Automation_AI-Assignment-Checker",
"updated_at": null,
"url": "https://github.com/deepvisionaiofficial/Automation_AI-Assignment-Checker"
},
{
  "categories": [
    "benchmark",
    "agentic"
  ]
}

```

use internal rubrics to compute retrieval score

Build · DeepSeek V4 Flash DeepSeek

D:\projects\vradar-test 187.3K (10%) · \$0.04 ctrl+ip commands

Figure 7: A candidate record with query-word matches and retrieval-score components for the agent to inspect when judging relevance.

Figure 8 shows the resulting related-work table, which the contributor judged complete enough to act on.

#	Work	Published	Qwen base model(s)	Credit-assignment focus	Agentic benchmarks used	Link
1	SRPO: Self-Reflective Policy Optimization for Long-Horizon Reasoning	2026-08-25	Qwen3-8B (also Qwen3-1.7B / 32B scaling)	Reflection-conditioned dense token-level signals convert sparse terminal reward into training signal	AIIME24, WebShop, ALFWorld, SWE-Bench-Lite	https://arxiv.org/abs/2608.23493 · https://github.com/Galleons2029/SRPO
2	ContextPilot: Teaching Agents for Proactive Context Management via Fine-grained RL	2026-08-28	Qwen3-8B, Qwen3-14B	Coarse-grained → action-level credit assignment over branched trajectories during agentic RL	InfBench (InfiniteBench), NovelQA, LongMemEval, BrowseComp+	https://arxiv.org/abs/2608.28476 · https://github.com/Tencent/ContextPilot
3	SkillGate: Training In-Policy Skill Selection in Long-Horizon Agents	2026-08-21	Qwen3.5-9B (SFT checkpoint, RL init)	"Selector credit starvation": partitions token support into two credit channels (outcome vs. action-local advantage)	5 agentic benchmarks incl. SkillsBench, SWE (16-candidate skill slate)	https://arxiv.org/abs/2608.18852
4	CIPO: Contextual Information Policy Optimization for Search Agents	2026-08-06	Qwen2.5-3B-Instruct, Qwen2.5-7B-Instruct	Dense turn-level credit (EALR) to evidence-using reasoning actions, combined with outcome reward	HotpotQA, 2WikiMultiHopQA, MuSiQue, Bamboogle + 3 OOD	https://arxiv.org/abs/2608.06128
5	MoRSE: Task-Oriented Multi-Agent System with Mixture of Role-Subtask Experts	2026-08-10	Qwen3-4B-Instruct (one of three backbones; also Llama-3.1-8B, Gemma-4-31B)	Hierarchical GRPO with two-layer credit assignment (isolates expert vs. routing quality)	Code-generation benchmarks (multi-agent), held-out task domains	https://arxiv.org/abs/2608.09251

Figure 8: Summary table of recent work on credit assignment in agentic training, assembled during the session.

Figure 9 shows a comparison table from an earlier benchmark-design effort by one contributor. It covers ten related benchmarks across six design dimensions, with each cell read from a source paper. The contributor reported that assembling it took more effort than any part of that project except producing the benchmark data. Benchmark Radar retrieves candidates for this kind of comparison and exposes the matching words and fields for inspection; the comparison itself requires reading the source evidence.

Bench Name	End-to-End Tasks	Fine-Grained Eval	Researcher-Quality Eval	Data Generation	Multi-Harness Eval	#Tasks
MLE-Bench (Chan et al. 2025)	✗	✓	✗	Transfer&Compose	✓	75
MLGym-Bench (Nathani et al. 2025)	✗	✓	✗	Automatic	✗	13
EXP-Bench (Kon et al. 2025)	✓	✗	✗	Automatic	✓	461
ResearchCodeBench (Hua et al. 2026)	✗	✓	✗	Transfer&Compose	✗	212
MLR-Bench (Chen et al. 2026)	✓	✗	✗	Automatic	✓	201
PaperBench (Starace et al. 2025)	✗	✓	✗	Transfer&Compose	✗	8316
AstaBench (Bragg et al. 2025)	✓	✓	✗	Transfer&Compose	✓	2400+
InnovatorBench (Wu et al. 2025)	✓	✗	✗	Transfer&Compose	✗	20
AIRS-Bench (Lupidi et al. 2026)	✗	✓	✗	Automatic	✓	20
COMPOSITE-Stem (Waters et al. 2026)	✓	✓	✗	Manual	✗	70
ScienceBoard (Sun et al. 2025)	✗	✓	✗	Manual	✗	169

Figure 9: A prior-art comparison table assembled manually for an earlier benchmark-design effort. Each row is one related benchmark and each column one design dimension, all read from the source papers by hand.

This case records one workflow combining Benchmark Radar with web search; it has no measured time or accuracy comparison.³ The summary table and session evidence are available in the project’s public [issue #492](#).

D Reproducibility, Access, and Citation

This revision uses the frozen [Benchmark Radar v0.11.0 release](#), software commit 8f46bbf, with a discovery cutoff of 2026-09-07. The release preserves the inputs already audited for this paper; later data is outside its scope. A fresh detached checkout passed the six required CI steps in order: linting, formatting checks, catalog normalization, classification, release construction, and tests. All 1,314 tests passed. The dated registry inputs remain the August snapshots registered in `data/leaderboard_snapshots.yml`; the September cutoff does not imply that those sources were recrawled on that day.

The data build normalizes the registry and model-report inputs into the shared catalog, then classifies discovery tracks and packages the catalog with checksums. Installed clients validate an update before activating it. The manuscript and its dated analysis files build independently of the software checkout. The project provides a DOI and Citation File Format record for citation [3, 8, 15].

Artifact	Canonical or permanent location
Archived paper (v0.9.0)	https://doi.org/10.5281/zenodo.22167102
Frozen data (v0.11.0)	https://github.com/ktwu01/benchmark-radar/releases/tag/v0.11.0
Source code	https://github.com/ktwu01/benchmark-radar
Dashboard	https://benchmark-radar.org/
Discovery observations (JSON)	https://benchmark-radar.org/data/radar.json
Benchmark catalog	https://benchmark-radar.org/data/benchmark-index.json
RSS	https://benchmark-radar.org/feed.xml
Citation metadata	https://github.com/ktwu01/benchmark-radar/blob/main/CITATION.cff

Table 7: Access and citation artifacts for Benchmark Radar.

The census reads `site/data/benchmark-index.json`, the detail shards under `site/data/benchmarks/`, and the shared document and model registries. Discovery counts

³Jiayu Wang contributed this worked use case and its supporting evidence.

come from the rebuilt `site/data/radar.json` and dated snapshots. Model reports and score YAML files are normalization inputs, not a separate population for the paper’s findings.

The paper repository contains `scripts/audit_catalog.py`, which validates IDs and counts, computes the census, and exports `catalog-data.tex` plus `evidence/catalog-audit.json`. The JSON includes every source-record key, measurement state, per-record model and document counts, citation IDs, and SHA-256 hashes of the inputs. The script checks score-observation uniqueness, record-to-shard identity, document-registry coverage, and reconciliation of scored and unscored records. Its `--check` mode compares a fresh calculation against the committed outputs. Figure 1 draws one linked dot per census record in source-key order.

The v0.11.0 release includes a frozen data archive and checksums matching the recorded audit inputs. To reproduce from source, check out software commit 8f46bbf, install its development dependencies, and run the six-step CI sequence. Run the software’s `scripts/export_report_figure_data.py` and its `--check` mode to refresh `figure-data.tex`. From the paper checkout, run `python scripts/audit_catalog.py /path/to/software`, then repeat with `--check`. The README records the commands. Finally, run `make arxiv`, compile the extracted package, and inspect the rebuilt PDF. Normal paper builds use the committed data and do not refresh the evidence.

The DOI in Table 7 identifies the archived v0.9.0 paper. This manuscript describes the v0.11.0 release at software commit 8f46bbf and the cutoff above. The dashboard continues to change; citations of its live counts need a retrieval date.